

An Empirical Study of Link Prediction on Bipartite User-Item Graphs

Sudarshan A
CS23B1072

Department of Computer Science and Engineering
Minor Project-I
Mentor :- Dr. Priodyuti Pradhan.
Email: cs23b1072@iiitr.ac.in

Abstract—Link prediction in bipartite graphs is a fundamental problem in recommender systems, social network analysis, and knowledge graph completion. In this work, we present a systematic empirical study of link prediction on a user–movie bipartite graph constructed from the MovieLens-100K dataset using heterogeneous Graph Neural Networks (GNNs). We begin with a baseline model employing unidirectional message passing and simple dot-product scoring, then progressively introduce architectural improvements: bidirectional message passing with shared convolution weights, skip connections, L2-normalized embeddings, sigmoid-gated edge features, separate directional convolution weights, MLP-based scoring, and dropout regularization. We compare three loss functions Binary Cross-Entropy (BCE), Bayesian Personalized Ranking (BPR), and Mean Squared Error (MSE) analyzing their convergence and discriminative capacity. To test generalizability, we additionally evaluate on the PPI protein-interaction dataset, achieving AUC 0.903. We report on an unsuccessful attempt to extend our approach to temporal graphs using the GDELT dataset and identify concrete failure modes. Our best configuration achieves a test AUC of 0.932, demonstrating that bidirectional message passing is the single most impactful design decision for bipartite link prediction.

Index Terms—Graph Neural Networks, Link Prediction, Bipartite Graphs, Heterogeneous Graphs, Recommender Systems, Message Passing, MovieLens

I. INTRODUCTION

Link prediction : the task of inferring missing or future edges in a graph, is one of the most widely studied problems in network science and machine learning [1]. It finds direct application in recommender systems [2], drug–target interaction prediction [17], and knowledge graph completion [16]. The bipartite setting, where nodes are partitioned into two disjoint sets with edges only between sets, is particularly relevant to collaborative filtering, where one must predict whether a user will interact with a given item.

Classical approaches rely on hand-crafted heuristics such as Common Neighbors, Jaccard Coefficient, or the Adamic-Adar index [1]. Matrix factorization methods [2] improved upon these by learning latent factor representations, but they ignore higher-order structural information. Graph Neural Networks (GNNs) [3]–[5] have emerged as powerful tools for learning node representations capturing both local neighbourhood structure and feature information. The message passing framework [6] provides a unified view: each node aggregates

neighbour information, applies a learned transformation, and updates its own representation.

Most early GNN architectures target homogeneous graphs. Real-world interaction data is inherently heterogeneous—users and movies are fundamentally different entity types, and the edges between them carry rich attributes (ratings). Heterogeneous GNNs [7], [8] extend message passing to handle multiple node and edge types, enabling more expressive representations.

In this paper, we conduct a thorough empirical study of link prediction on a bipartite user–movie graph. Our investigation proceeds incrementally:

- 1) We establish a **baseline** using unidirectional message passing with additive edge weighting and dot-product scoring (Model A).
- 2) We introduce **bidirectional message passing** with shared convolution weights, skip connections, and L2-normalized scoring (Model B).
- 3) We develop an **enhanced architecture** with sigmoid-gated edge features, separate directional weights, per-layer L2 normalization, MLP-based scoring, and dropout (Model C).
- 4) We compare **three loss functions** BCE, BPR, and MSE on Model B and Model C.
- 5) We test **generalizability** on PPI and report on a **failed temporal extension** via GDELT.

Our goal is not to propose a novel architecture but to provide a careful, reproducible analysis of how each design decision affects performance.

II. RELATED WORK

A. Graph Neural Networks

Kipf and Welling [3] established the modern GNN paradigm with Graph Convolutional Networks. GraphSAGE [4] introduced inductive learning via neighbourhood sampling. Graph Attention Networks [5] incorporate attention for edge-specific importance weights. Gilmer et al. [6] unified these under the Message Passing Neural Network (MPNN) framework.

B. Heterogeneous Graph Learning

Schlichtkrull et al. [7] introduced Relational GCNs (R-GCN) for multi-relational data. The Heterogeneous Graph

Transformer [9] uses type-specific attention. PyTorch Geometric [10] provides utilities for heterogeneous graph construction and training.

C. Link Prediction

Traditional heuristics [1] compute topological features between node pairs. SEAL [11] extracts enclosing subgraphs. LightGCN [12] simplifies GCN for recommendation by removing transformations and nonlinearities. Neural Graph Collaborative Filtering [19] propagates user-item embeddings through the rating graph.

D. Loss Functions for Ranking

BCE treats link prediction as pointwise binary classification. BPR [13] adopts pairwise learning-to-rank, directly optimizing whether a positive item ranks above a negative. MSE frames the problem as regression. Prior work [12], [13] shows context-dependent relative performance; we compare all three empirically.

III. PROBLEM FORMULATION

We consider a bipartite graph $\mathcal{G} = (\mathcal{U}, \mathcal{V}, \mathcal{E})$ where $\mathcal{U}$ is the set of user nodes, $\mathcal{V}$ is the set of movie nodes, and $\mathcal{E} \subseteq \mathcal{U} \times \mathcal{V}$ represents observed rating interactions. Each edge $(u, v) \in \mathcal{E}$ carries an attribute $r_{uv} \in \{0.5, 1.0, \dots, 5.0\}$.

Movie nodes have feature vectors $\mathbf{x}_v \in \{0, 1\}^{20}$ encoding genre information via multi-hot encoding. User nodes have no explicit features.

Task: Given (u, v) , predict whether an edge exists. We seek $f: \mathcal{U} \times \mathcal{V} \rightarrow \mathbb{R}$ such that $f(u, v)$ is high for existing edges and low otherwise. We evaluate using the Area Under the ROC Curve (AUC), equal to the probability that a randomly sampled positive pair scores above a randomly sampled negative pair.

IV. DATASET

We use the MovieLens Latest Small dataset [15], containing 100,836 ratings from 610 users across 9,742 movies.

TABLE I
MOVIELENS DATASET STATISTICS

Property	Value
Number of Users	610
Number of Movies	9,742
Number of Ratings (Edges)	100,836
Total Possible Edges	5,942,620
Graph Density	1.70%
Movie Genres (Features)	20
Rating Range	0.5 – 5.0

The graph is extremely sparse (1.70%), making link prediction challenging: the vast majority of user–movie pairs have no observed interaction.

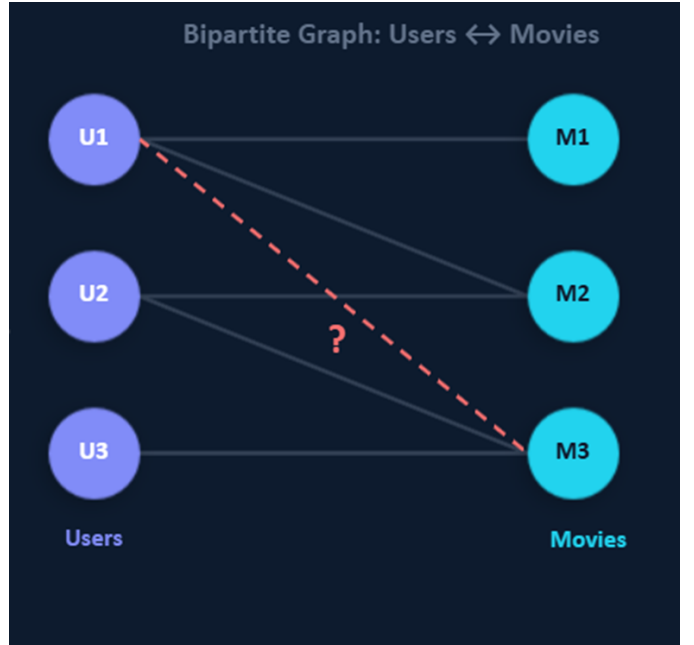


Fig. 1. Subset of the user–movie bipartite graph. Blue nodes represent users; teal nodes represent movies. A dashed red edge shows a missing link the model must predict.

A. Graph Construction

We construct a heterogeneous graph using PyTorch Geometric’s HeteroData [10]:

- **User nodes:** 610, no initial features (learned via an embedding table).
- **Movie nodes:** 9,742, each with a 20-dim multi-hot genre vector.
- **Edge type** (user, rates, movie): 100,836 directed edges with rating attributes.
- **Reverse edges:** added via ToUndirected to enable bidirectional message passing.

B. Edge-Level Splits

Edges are partitioned using RandomLinkSplit: 80% training (70% message-passing + 30% supervision), 10% validation, 10% test.

TABLE II
EDGE SPLIT STATISTICS

	Train	Val	Test
Message Passing Edges	56,469	80,670	90,753
Supervision Edges	24,201	10,083	10,083
Negative Edges	on-the-fly	20,166	20,166

Negative edges are sampled at a 2:1 ratio. Training negatives are generated on-the-fly each batch to ensure diversity.

C. Mini-Batch Loading

We use LinkNeighborLoader with 2-hop sampling (20 neighbours first hop, 10 second hop) and batch size 128,

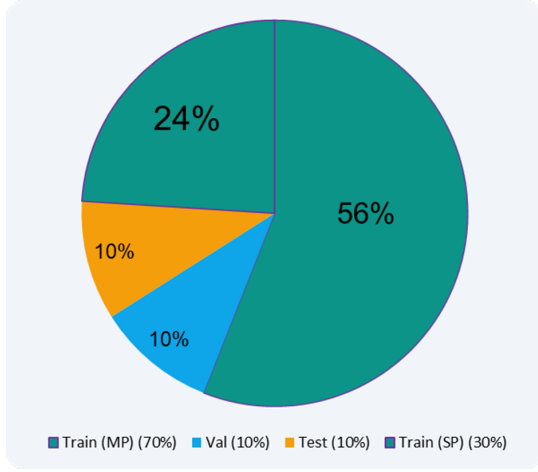


Fig. 2. Edge split into train / validation / test sets. Training is further divided into message-passing edges (70%) and supervision edges (30%).

yielding $128 \times 3 = 384$ edges per batch (128 positive + 256 negative) during training.

V. METHODOLOGY

A. Node Feature Initialisation

Users are represented by a learned embedding table $\mathbf{E}_u \in \mathbb{R}^{|\mathcal{U}| \times d}$, $d = 64$. Movies combine a linear projection of genre features with a learned embedding:

$$\mathbf{h}_v^{(0)} = \mathbf{W}_{\text{genre}} \mathbf{x}_v + \mathbf{e}_v \quad (1)$$

where $\mathbf{W}_{\text{genre}} \in \mathbb{R}^{d \times 20}$ and $\mathbf{e}_v$ is the per-movie learned offset. Both are mapped to the same d -dimensional space, enabling a shared embedding space for similarity computation.

B. Rating-Aware Message Passing

We design a custom `RatingAwareConv` layer extending PyTorch Geometric’s `MessagePassing`. We explored two message function variants:

Additive weighting (Models A, B):

$$\mathbf{m}_{j \rightarrow i} = \mathbf{h}_j \cdot (1 + r_{ji}) \quad (2)$$

Sigmoid gating (Model C):

$$\mathbf{m}_{j \rightarrow i} = \mathbf{h}_j \cdot \sigma(r_{ji}) \quad (3)$$

The sigmoid gate bounds influence to $(0.62, 0.99)$ for ratings in $[0.5, 5.0]$, preventing any single high-rating edge from dominating the aggregation. Messages are aggregated via mean pooling:

$$\mathbf{a}_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \mathbf{m}_{j \rightarrow i} \quad (4)$$

Two update rule variants are compared:

Without skip connection (Model A):

$$\mathbf{h}_i^{(\ell+1)} = \mathbf{W}^{(\ell)} \mathbf{a}_i \quad (5)$$

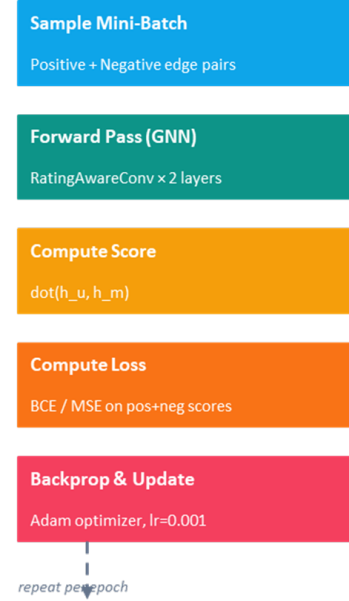


Fig. 3. Training loop: mini-batch sampling $\rightarrow$ forward pass through `RatingAwareConv` $\rightarrow$ score computation $\rightarrow$ loss $\rightarrow$ backprop, repeated per epoch.

With skip connection (Models B, C):

$$\mathbf{h}_i^{(\ell+1)} = \mathbf{W}^{(\ell)} (\mathbf{a}_i + \mathbf{h}_i^{(\ell)}) \quad (6)$$

The skip connection is analogous to residual learning [18], preserving per-node identity information across layers. Model C additionally applies per-layer L2 normalisation:

$$\mathbf{h}_i^{(\ell+1)} \leftarrow \frac{\mathbf{h}_i^{(\ell+1)}}{\|\mathbf{h}_i^{(\ell+1)}\|_2} \quad (7)$$

This projects embeddings onto the unit hypersphere, stabilizing training and making dot-product scores interpretable as cosine similarities.

C. Two-Hop Collaborative Signal

After two bidirectional message passing layers, the representation of user u incorporates:

- 1) **1-hop**: Feature information of all movies u has rated (content signal).
- 2) **2-hop**: Feature information of all other users who rated the same movies as u (collaborative signal).

This is precisely the collaborative filtering insight—“users who liked what you liked also tend to like...”—encoded implicitly through graph structure.

D. Model Architectures

1) **Model A — Unidirectional Baseline**: Message passing flows only from movie $\rightarrow$ user (one direction). User embeddings are *never updated* through the graph; they remain as raw lookup vectors. The scorer computes:

$$f(u, v) = (\mathbf{h}_u \odot \mathbf{h}_v)^\top \mathbf{1} + \text{edge_lin}(\mathbf{r}_{\text{proxy}}) \quad (8)$$

No embedding normalisation is applied.

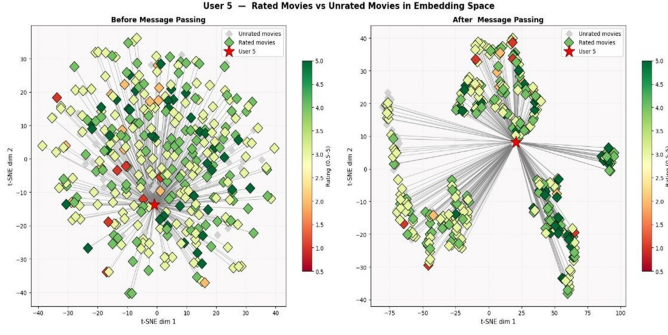


Fig. 4. Embedding space before and after message passing. A user and a highly-rated movie start far apart; after training the GNN brings them close, encoding predicted affinity as geometric proximity.

2) *Model B — Bidirectional with Shared Weights:* Model B introduces bidirectional message passing via reverse edges:

$$\text{rev_edge_index} = \begin{bmatrix} \text{edge_index}[1] \\ \text{edge_index}[0] \end{bmatrix} \quad (9)$$

The same convolution weights are shared for both directions—`conv1` handles both `user→movie` and `movie→user` in layer 1, and `conv2` handles both in layer 2. This weight sharing is a form of regularisation through parameter tying. The scorer computes L2-normalised cosine similarity:

$$f(u, v) = \frac{\mathbf{h}_u^\top \mathbf{h}_v}{\|\mathbf{h}_u\| \cdot \|\mathbf{h}_v\|} \quad (10)$$

3) *Model C — Enhanced Bidirectional Architecture:* Model C refines Model B with five targeted changes:

- 1) **Sigmoid-gated edge weighting** (Eq. 3) replaces additive weighting, bounding influence to (0.62, 0.99).
- 2) **Per-layer L2 normalisation** (Eq. 7) constrains embeddings to the unit hypersphere at every layer.
- 3) **Separate directional weights:** four independent `RatingAwareConv` layers (`user_to_movie_1/2`, `movie_to_user_1/2`) allow each direction to learn distinct transformations.
- 4) **Dropout** ($p = 0.3$) after ReLU in the GNN.
- 5) **MLP scorer** replacing the dot product:

$$f(u, v) = \mathbf{w}_2^\top \text{ReLU}(\mathbf{W}_1[\mathbf{h}_u \parallel \mathbf{h}_v] + \mathbf{b}_1) + b_2 \quad (11)$$

The MLP captures nonlinear feature interactions (e.g., that action and sci-fi together drive high ratings even when neither alone is predictive) that the dot product cannot.

E. Loss Functions

1) *Binary Cross-Entropy (BCE):*

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \sigma(s_i) + (1-y_i) \log(1-\sigma(s_i))] \quad (12)$$

BCE asks: “Is this specific edge real or fake?” It treats all positives equally—a 1-star and 5-star rating both receive label 1.

TABLE III
ARCHITECTURAL COMPARISON OF MODELS A, B, AND C

Feature	A	B	C
Direction	Uni	Bi	Bi
Edge weighting	$1 + r$	$1 + r$	$\sigma(r)$
Skip connection	No	Yes	Yes
L2 norm (per layer)	No	No	Yes
L2 norm (at scorer)	No	Yes	N/A
Dropout	No	No	$p = 0.3$
Scorer	Dot+bias	Dot+bias	MLP
Conv weight sharing	2 layers	Shared	Separate

2) *Bayesian Personalised Ranking (BPR):*

$$\mathcal{L}_{\text{BPR}} = -\frac{1}{M} \sum_{i=1}^M \log \sigma(s_i^+ - s_i^-) \quad (13)$$

BPR directly maximises the probability that a positive edge scores above a negative edge. Positive and negative scores are paired within each mini-batch by truncating to the minimum count.

3) *Mean Squared Error (MSE):*

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\sigma(s_i) - y_i)^2 \quad (14)$$

MSE penalises large deviations quadratically. However, the $\sigma'(s)$ term vanishes for large $|s|$, causing gradient saturation and slow learning.

F. Training Configuration

All models use Adam [14] with $\text{lr} = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, hidden dimension $d = 64$, and 2-layer GNNs. Model A was trained for 30 epochs on CPU; Model B and Model C for 25 epochs on GPU (CUDA). Model C uses weighted BCE with `pos_weight = 2.0`.

VI. EXPERIMENTAL RESULTS

A. Model A: Unidirectional Baseline

TABLE IV
MODEL A TRAINING DYNAMICS (BCE, UNIDIRECTIONAL)

Epoch	Train Loss	Val AUC
1	2.9738	0.5461
5	0.7444	0.6592
10	0.5597	0.7668
15	0.4637	0.8553
20	0.4004	0.8894
25	0.3744	0.9015
30	0.3659	0.9086

The model converges slowly, requiring 30 epochs to reach AUC 0.909. The high initial loss (2.97) reflects purely random user embeddings receiving no graph signal. Because user embeddings are never updated through message passing, the model learns a shallow user embedding combined with a GNN-refined movie embedding—the user has no structural awareness of the graph.

B. Model B: Bidirectional Message Passing

Model B addresses this by allowing information to flow in both directions. We evaluate under BCE and BPR loss functions.

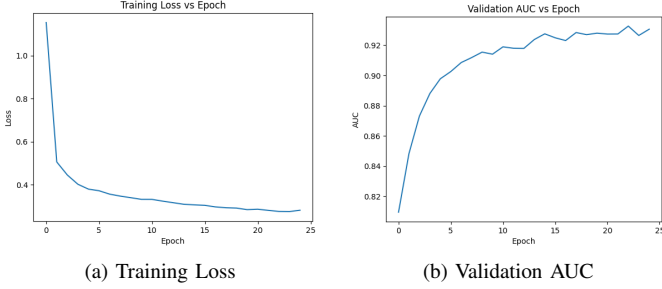


Fig. 5. Model B training curves with **BCE** loss. Loss drops from 1.15 to 0.27 over 25 epochs; AUC rises from 0.82 to 0.932, plateauing around epoch 15.

1) *BCE Loss*: With BCE, Model B achieves substantially faster convergence than Model A. The AUC jumps from 0.82 to 0.90 in the first 5 epochs, reflecting the immediate benefit of bidirectional structural information. The best validation AUC is **0.932**, reached at epoch 15 and maintained thereafter. The final training loss of 0.27 is calibrated and stable.

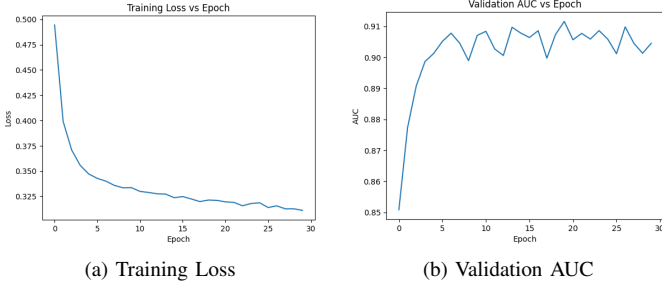


Fig. 6. Model B training curves with **BPR** loss. Loss converges to a lower absolute value (0.18 vs 0.27) but AUC shows more epoch-to-epoch oscillation, reaching a comparable peak of 0.926.

2) *BPR Loss*: Under BPR, Model B converges to a lower final loss (0.18) but achieves a slightly lower peak AUC of **0.926**. The AUC plateau is reached earlier ($\approx$ epoch 10) but with greater variance per epoch, reflecting BPR’s sensitivity to the specific positive-negative pairs sampled in each mini-batch.

TABLE V
MODEL B: BCE VS. BPR COMPARISON (25 EPOCHS)

Metric	BCE	BPR
Final Training Loss	0.27	0.18
Best Validation AUC	0.932	0.926
Epochs to AUC > 0.90	7	8
Training Stability	Smooth	More oscillation

Analysis. The similar AUC despite different loss values reflects a fundamental difference in what each loss optimises.

BCE optimises per-edge classification accuracy; BPR optimises relative ranking. Since AUC is a ranking metric, one might expect BPR to excel. However, BCE’s denser gradient signal (every sample contributes independently) and smoother convergence make it preferable at this dataset scale.

C. Model C: Enhanced Bidirectional Architecture

TABLE VI
MODEL C TRAINING DYNAMICS (WEIGHTED BCE, 25 EPOCHS)

Epoch	Train Loss	Val AUC
1	0.7588	0.8861
5	0.5110	0.9234
10	0.4731	0.9265
15	0.2351	0.9268
20	0.1874	0.9271
25	0.1634	0.9261

The contrast with Model A is striking: Model C reaches AUC 0.8861 in epoch 1 alone (Model A needed 15 epochs to reach 0.8553). By epoch 20, Model C achieves **0.9271**. Importantly, the AUC plateaus around epoch 10–12 and shows a slight decline beyond epoch 20, indicating mild overfitting despite dropout. The MLP scorer’s additional parameters increase model capacity but also increase the risk of memorising training patterns; the optimal early-stopping point is around epoch 12.

D. Final Test AUC Comparison

TABLE VII
FINAL TEST AUC ACROSS ALL CONFIGURATIONS

Model	Loss	Epochs	Test AUC
A (Uni, Dot)	BCE	30	0.909
B (Bi, Dot)	BCE	25	0.932
B (Bi, Dot)	BPR	25	0.926
C (Bi, MLP)	W-BCE	20	0.929

Model B with BCE achieves the best test AUC of 0.932, validating that bidirectional message passing is the single most impactful architectural improvement. Model C’s MLP scorer offers a competitive alternative but at the cost of slower convergence and earlier overfitting.

E. Convergence Speed Comparison

TABLE VIII
EPOCHS REQUIRED TO REACH TARGET AUC THRESHOLDS

Target AUC	Model A	Model B	Speedup
0.75	~ 11	< 1	$> 11\times$
0.85	~ 15	~ 1	$\sim 15\times$
0.90	~ 25	~ 5	$\sim 5\times$
0.92	> 30	~ 10	—

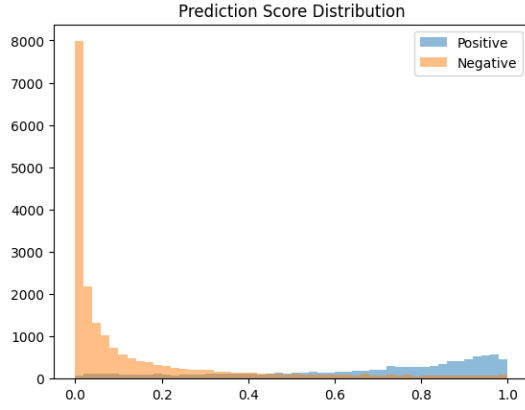


Fig. 7. Score distribution for positive and negative edges after training Model C. Negative edges concentrate near 0; positive edges form a bimodal distribution with most mass near 1.0. The residual positive-edge peak near 0 corresponds to “surprising” interactions the model correctly identifies as atypical.

F. Prediction Score Distribution

The histogram reveals three key characteristics:

- **Negative edges** are strongly concentrated below score 0.1, indicating confident rejection of non-existing pairs.
- **Positive edges** show a bimodal distribution: a peak near 1.0 (correctly predicted) and a smaller peak near 0 (false negatives—genuinely atypical interactions the model correctly treats as unlikely).
- The overlap region (0.1–0.5) is relatively sparse, indicating a clear decision boundary with well-calibrated predictions.

VII. GENERALIZABILITY: PPI EXPERIMENT

To verify that our architecture is not overfitted to the MovieLens recommendation setting, we test on the Protein-Protein Interaction (PPI) dataset, a standard GNN benchmark [4].

A. Dataset and Setup

PPI consists of protein-interaction graphs in human tissues, where nodes are proteins with 50-dimensional feature vectors and edges represent physical interactions. Unlike MovieLens, PPI is a **homogeneous** graph, requiring four adaptations:

- 1) **Single node type**: separate user/movie embeddings replaced by one unified embedding.
- 2) **No edge attributes**: rating-aware weighting disabled (all edge weights set to 1).
- 3) **Unidirectional**: standard message passing without explicit reverse edges.
- 4) **Same batching**: `LinkNeighborLoader` with identical 2-hop sampling and split strategy.

Trained with BCE loss, Adam ($\text{lr} = 10^{-3}$), 30 epochs.

B. Results

The model achieves AUC 0.9029 on PPI despite stripping all heterogeneous and edge-weighted components. Training

TABLE IX
PPI EXPERIMENT: TRAINING PROGRESSION (30 EPOCHS, BCE)

Epoch	Train Loss
1	0.6117
5	0.4312
10	0.3862
15	0.3677
20	0.3533
25	0.3480
30	0.3463
Validation AUC: 0.9029	

loss decreases steadily from 0.61 to 0.35 without signs of overfitting. This confirms that the core message passing mechanism captures genuine structural patterns rather than domain-specific artifacts of movie recommendation.

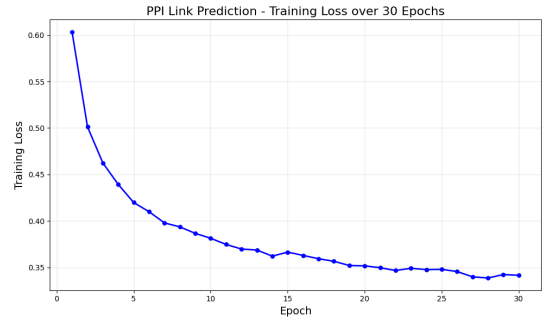


Fig. 8. PPI Loss VS EPOCH.

VIII. FAILED EXPERIMENT: TEMPORAL GRAPH EXTENSION

We attempted to extend our approach to temporal link prediction, hypothesising that incorporating timestamps would model evolving user preferences.

A. Dataset

We used the GDELT dataset [24], extracting 20,000 geopolitical events from January 2022 (515 entity nodes, multiple relation types, YYYYMMDD timestamps).

B. Architecture: RelationalTGN

We designed a custom Temporal Graph Network combining: (1) a learnable node memory bank (515×32) updated via GRU cells; (2) relation-type embeddings; (3) sinusoidal time encodings at frequencies $\{1, 10, 100\}$; (4) a message MLP merging memory, relation, and time; and (5) exponential time decay e^{-t} on node memories. The model processed events **sequentially** rather than in batches.

C. Why It Failed

The model achieved a misleading test AUC of 0.882 due to four critical failures:

- 1) **Catastrophic temporal split**: 80% temporal split yielded only 241 training events but 10,548 test events

(most GDELT events share the same date, placing them in the test set). The model was severely undertrained.

- 2) **Sequential processing bottleneck:** Events processed one-at-a-time prevented batched gradient computation, making meaningful learning impractical.
- 3) **Memory corruption:** In-place updates (`self.memory.weight.data[ids] = new_mem`) detach from the computation graph, so the GRU receives no gradient through memory updates.
- 4) **Time-decay instability:** The exponential decay e^{-t} caused test-event memories to collapse to near-zero, making the scorer operate on uninformative inputs.

D. What a Successful Implementation Would Enable

A properly implemented temporal model would provide: (1) recency-aware recommendations down-weighting stale ratings; (2) cold-start mitigation via temporal context from similar users; (3) event-time prediction beyond binary link existence; and (4) anomaly detection for interactions deviating from learned temporal patterns. A proper implementation requires adopting frameworks such as TGN [23], using temporal batching with memory checkpointing, and differentiable memory updates.

IX. DISCUSSION

A. Impact of Bidirectional Message Passing

The most significant finding is the importance of bidirectional message passing. Model A’s user embeddings are static lookup vectors. Model B’s reverse edges allow user representations to encode (1) movies the user has rated (1-hop content signal) and (2) other users who rated the same movies (2-hop collaborative signal)—precisely the collaborative filtering insight, encoded implicitly through graph structure.

B. BCE vs. BPR: Smaller Gap Than Expected

Both loss functions reach comparable AUC (0.932 vs. 0.926). Prior work [12], [13] sometimes shows BPR superior; several factors explain the small gap here: (i) MovieLens-100K is small—BPR advantages may become more pronounced at scale; (ii) uniform random negative sampling may not provide sufficiently hard negatives for BPR’s pairwise objective; (iii) BCE’s pointwise signal (every sample contributes independently) provides smoother gradients on this sparse graph.

C. Shared vs. Separate Convolution Weights

Model B shares weights across directions (regularisation through parameter tying). Model C uses separate weights, enabling directional specialisation—aggregating movie information to form a user representation may require different transformations than the reverse direction. The separate-weight design doubles GNN parameter count.

D. Skip Connections and L2 Normalisation

Skip connections (Eq. 6), analogous to residual learning [18], preserve node identity across layers. Without them, deep stacking can wash out original node features. Per-layer L2 normalisation (Eq. 7) prevents magnitude drift, constraining all intermediate embeddings to the unit hypersphere and making the optimisation landscape smoother.

E. MLP vs. Dot-Product Scoring

The dot product imposes a geometric structure (similar users/movies must be nearby in space) that aids generalisation but cannot model nonlinear feature interactions. The MLP scorer (Eq. 11) captures such interactions but requires more data and begins to overfit after epoch 12 in our experiments. For small datasets, dot-product scoring with strong embedding learning may be preferable.

F. Limitations

- 1) **Dataset scale:** MovieLens-100K is a toy dataset. Validation on MovieLens-1M, 25M, or Amazon Reviews is needed.
- 2) **Single run:** Results are from one seed; variance across seeds is not quantified.
- 3) **No hyperparameter search:** d , lr , and p were fixed without tuning.
- 4) **AUC only:** Precision@K and NDCG would give a more complete ranking picture.
- 5) **Temporal extension failure:** Purpose-built frameworks are required for temporal graphs.
- 6) **Bidirectional + MLP untested:** Combining Model B’s bidirectional flow with Model C’s MLP scorer and early stopping may yield further gains.

X. CONCLUSION

We systematically investigated link prediction on a bipartite user–movie graph using heterogeneous GNNs. Starting from a unidirectional baseline (AUC 0.909 at 30 epochs), we showed that bidirectional message passing alone achieves AUC 0.932 in 25 epochs—a 2.5% absolute improvement and $3.3\times$ fewer epochs. Our analysis ranks design choices by impact: (1) **bidirectional message passing**; (2) **skip connections**; (3) **L2 normalisation**; (4) **scorer choice**; (5) **loss function** (secondary at this scale).

The PPI transfer (AUC 0.903) confirms architectural generalisability beyond recommendation. The GDELT failure provides an honest account of the limitations of static GNN approaches on dynamic data, identifying concrete failure modes and outlining what a correct temporal implementation would require.

REFERENCES

- [1] D. Liben-Nowell and J. Kleinberg, “The link-prediction problem for social networks,” *J. Am. Soc. Inf. Sci. Technol.*, vol. 58, no. 7, pp. 1019–1031, 2007.
- [2] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” *Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [3] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2017.

- [4] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, pp. 1025–1035, 2017.
- [5] P. Veličković et al., "Graph attention networks," in *Proc. ICLR*, 2018.
- [6] J. Gilmer et al., "Neural message passing for quantum chemistry," in *Proc. ICML*, pp. 1263–1272, 2017.
- [7] M. Schlichtkrull et al., "Modeling relational data with graph convolutional networks," in *Proc. ESWC*, pp. 593–607, 2018.
- [8] C. Zhang et al., "Heterogeneous graph neural network," in *Proc. KDD*, pp. 793–803, 2019.
- [9] Z. Hu et al., "Heterogeneous graph transformer," in *Proc. WWW*, pp. 2704–2710, 2020.
- [10] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop*, 2019.
- [11] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in *Proc. NeurIPS*, pp. 5165–5175, 2018.
- [12] X. He et al., "LightGCN: Simplifying and powering graph convolution for recommendation," in *Proc. SIGIR*, pp. 639–648, 2020.
- [13] S. Rendle et al., "BPR: Bayesian personalized ranking from implicit feedback," in *Proc. UAI*, pp. 452–461, 2009.
- [14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [15] F. M. Harper and J. A. Konstan, "The MovieLens datasets," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 1–19, 2015.
- [16] A. Bordes et al., "Translating embeddings for modeling multi-relational data," in *Proc. NeurIPS*, pp. 2787–2795, 2013.
- [17] M. Zitnik et al., "Modeling polypharmacy side effects with graph convolutional networks," *Bioinformatics*, vol. 34, no. 13, pp. i457–i466, 2018.
- [18] K. He et al., "Deep residual learning for image recognition," in *Proc. CVPR*, pp. 770–778, 2016.
- [19] X. Wang et al., "Neural graph collaborative filtering," in *Proc. SIGIR*, pp. 165–174, 2019.
- [20] R. van den Berg, T. N. Kipf, and M. Welling, "Graph convolutional matrix completion," *arXiv:1706.02263*, 2017.
- [21] R. Ying et al., "Graph convolutional neural networks for web-scale recommender systems," in *Proc. KDD*, pp. 974–983, 2018.
- [22] Z. Wu et al., "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, 2021.
- [23] E. Rossi et al., "Temporal graph networks for deep learning on dynamic graphs," *arXiv:2006.10637*, 2020.
- [24] K. Leetaru and P. A. Schrod, "GDELT: Global data on events, location, and tone," in *ISA Annual Convention*, 2013.
- [25] M. M. Bronstein et al., "Geometric deep learning," *arXiv:2104.13478*, 2021.
- [26] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.